INFORME PRÁCTICA SOCKET- MALANCA

El presente proyecto consiste en el desarrollo de una aplicación distribuida basada en la arquitectura **Cliente-Servidor** utilizando el lenguaje de programación Java. El objetivo principal es establecer una conexión bidireccional mediante el protocolo **TCP/IP**, permitiendo que un cliente envíe cadenas de texto y el servidor las procese para devolver únicamente las consonantes presentes en dichas cadenas.

La solución se basa en sockets de flujo:

- **Servidor:** Utiliza la clase `ServerSocket` para escuchar peticiones en el puerto **5000**. Funciona de manera **iterativa** (monohilo), atendiendo a un cliente a la vez de forma secuencial.
- **Cliente:** Utiliza la clase `Socket` para conectarse al `localhost`. Permite la entrada de datos por consola mediante la clase `Scanner`.

El servidor implementa una lógica flexible para la recepción de comandos. Se programó para que el usuario pueda interactuar de dos maneras:

1. **Comando Explícito:** `CONSONANTES <palabra>`
2. **Texto Directo:** Simplemente enviando la palabra.

Es importante destacar que este diseño es **monohilo**. El hilo principal (`main thread`) gestiona tanto la aceptación de la conexión como el intercambio de mensajes. Esto implica que el servidor es bloqueante: hasta que el cliente actual no cierra la sesión (enviando "SALIR"), el servidor no puede procesar a un nuevo cliente.

```java
// interpreta el mensaje del cliente y decide qué hacer
private static String procesarMensaje(String mensaje) {
    // divide el mensaje en dos partes: el comando y el resto
    String[] partes = mensaje.trim().split("\\s+", 2);
    String comando = partes[0].toUpperCase();

    // el cliente escirbe CONSONANTES y luego la palabra
    if (comando.equals("CONSONANTES")) {
        // Verifica que haya una palabra después del comando
        if (partes.length < 2 || partes[1].trim().isEmpty()) {
            return "ERROR: Debes escribir una palabra. Ej: CONSONANTES programacion";
        }
        return obtenerConsonantes(partes[1].trim());
    }

    // el cliente escribe una palabra sin el CONSONANTES adelante
    if (partes.length == 1) {
        return obtenerConsonantes(mensaje.trim());
    }

    // cualquer error sobre la palabra
    return "Comando no reconocido. Usá CONSONANTES <palabra>, o escribí solo la palabra.";
}
```

```java
// Analiza una palabra y devuelve sus consonantes
private static String obtenerConsonantes(String palabra) {
    // Define todas las vocales (con y sin tilde) para poder excluirlas
    String vocales = "aeiouáéíóúüAEIOUÁÉÍÓÚÜ";

    // Acumula todas las consonantes encontradas, incluyendo repetidas
    StringBuilder todasLasConsonantes = new StringBuilder();

    // Guarda las consonantes únicas respetando el orden de aparición
    // LinkedHashSet elimina duplicados pero mantiene el orden
    LinkedHashSet<Character> unicas = new LinkedHashSet<>();

    int cantidad = 0;

    // Recorre cada letra de la palabra
    for (char c : palabra.toCharArray()) {
        // Solo procesa letras que NO sean vocales
        if (Character.isLetter(c) && vocales.indexOf(c) == -1) {
            todasLasConsonantes.append(c).append(" ");
            unicas.add(Character.toLowerCase(c)); // guarda en minúscula para no repetir Ll,
            cantidad++;
        }
    }

    // Si no encontró ninguna consonante, avisa
    if (cantidad == 0) {
        return "La palabra '" + palabra + "' no tiene consonantes.";
    }

    // Devuelve todas las consonantes, las únicas y el total
    return "Consonantes de '" + palabra + "': " + todasLasConsonantes.toString().trim()
            + "  |  únicas: " + unicas
            + "  |  total: " + cantidad;
}
```

El cliente es el encargado de la interfaz con el usuario. Su flujo de control es:

1. Establecer conexión con el puerto 5000.
2. Mostrar los mensajes de bienvenida enviados por el servidor.
3. Entrar en un bucle de lectura/escritura donde:
    a. Lee texto desde la consola.
    b. Lo envía mediante un `PrintWriter`.
    c. Espera y muestra la respuesta procesada.
4. Cerrar los flujos y el socket al detectar la palabra clave "SALIR".

```java
try (
    // Crea la conexión con el servidor en el puerto 5000
    Socket socket = new Socket(HOST, PUERTO);

    // entrada: para leer los mensajes que llegan del servidor
    BufferedReader entrada = new BufferedReader(new InputStreamReader(socket.getInputStream()));

    // salida: para enviar mensajes al servidor
    PrintWriter salida = new PrintWriter(socket.getOutputStream(), true);

    // teclado: para leer lo que escribe el usuario en la consola
    Scanner teclado = new Scanner(System.in)
) {
    System.out.println("Conexión establecida!\n");

    // Lee y muestra los mensajes de bienvenida que manda el servidor al conectarse
    String lineaBienvenida;
    while ((lineaBienvenida = entrada.readLine()) != null) {
        System.out.println("[SERVIDOR]: " + lineaBienvenida);

        // El servidor manda "---..." como última línea del saludo, ahí paramos
        if (lineaBienvenida.startsWith("---")) {
            break;
        }
    }
```

```java
    // Bucle principal: el usuario escribe, el servidor responde, y así sucesivamente
    while (true) {
        System.out.print("Vos: ");

        // Espera a que el usuario escriba algo y presione Enter
        String mensaje = teclado.nextLine();

        // Si el usuario presionó Enter sin escribir nada, ignora y vuelve a pedir
        if (mensaje.trim().isEmpty()) {
            continue;
        }

        // Envía el mensaje al servidor
        salida.println(mensaje);

        // Lee la respuesta del servidor y la muestra
        String respuesta = entrada.readLine();
        System.out.println("[SERVIDOR]: " + respuesta);

        // Si el usuario mandó SALIR, cierra el programa
        if (mensaje.equalsIgnoreCase("SALIR")) {
            System.out.println("[CLIENTE] Conexión cerrada. ¡Hasta luego!");
            break;
        }
    }
```

PRUEBAS DE FUNCIONAMIENTO

Cliente colocando CONSONANTES adelante de la palabra

```
run:
=== CLIENTE INICIADO ===
Conectando al servidor localhost:5000...
Conexión establecida!

[SERVIDOR]: Bienvenido al Servidor de Consonantes!
[SERVIDOR]: Comandos disponibles:
[SERVIDOR]:   CONSONANTES <palabra>  -> lista las consonantes de la palabra
[SERVIDOR]:   SALIR                  -> cierra la conexión
[SERVIDOR]: ---------------------------------------------------
Vos: CONSONANTES computadora
[SERVIDOR]: Consonantes de 'computadora': c m p t d r  |  únicas: [c, m, p, t, d, r]  |  total: 6
Vos:
```

Cliente colocando sólo la palabra

```
run:
=== CLIENTE INICIADO ===
Conectando al servidor localhost:5000...
Conexión establecida!

[SERVIDOR]: Bienvenido al Servidor de Consonantes!
[SERVIDOR]: Comandos disponibles:
[SERVIDOR]:   CONSONANTES <palabra>  -> lista las consonantes de la palabra
[SERVIDOR]:   SALIR                  -> cierra la conexión
[SERVIDOR]: ---------------------------------------------------
Vos: computadora
[SERVIDOR]: Consonantes de 'computadora': c m p t d r  |  únicas: [c, m, p, t, d, r]  |  total: 6
Vos: programación
[SERVIDOR]: Consonantes de 'programaci□n': p r g r m c n  |  únicas: [p, r, g, m, c, n]  |  total: 7
Vos:
```

Comando de salida

```
run:
=== CLIENTE INICIADO ===
Conectando al servidor localhost:5000...
Conexión establecida!

[SERVIDOR]: Bienvenido al Servidor de Consonantes!
[SERVIDOR]: Comandos disponibles:
[SERVIDOR]:   CONSONANTES <palabra>  -> lista las consonantes de la palabra
[SERVIDOR]:   SALIR                  -> cierra la conexión
[SERVIDOR]: ---------------------------------------------------
Vos: computadora
[SERVIDOR]: Consonantes de 'computadora': c m p t d r  |  únicas: [c, m, p, t, d, r]  |  total: 6
Vos: programación
[SERVIDOR]: Consonantes de 'programaci□n': p r g r m c n  |  únicas: [p, r, g, m, c, n]  |  total: 7
Vos: SALIR
[SERVIDOR]: Hasta luego! Cerrando conexión...
[CLIENTE] Conexión cerrada. ¡Hasta luego!
BUILD SUCCESSFUL (total time: 25 seconds)
```

El sistema cumple con los requisitos de comunicación de red solicitados. La implementación de Sockets en Java permite un manejo robusto de los flujos de datos a través de `BufferedReader` y `PrintWriter`. Si bien el servidor actual es iterativo, la estructura modular del código permitiría escalar fácilmente a un modelo multihilo en el futuro para dar soporte a múltiples conexiones simultáneas.